超星·期刊

利用统计数据并行蒙特卡罗树搜索算法的中国象棋博弈

朱 舟,闵华松
武汉科技大学 信息科学与工程学院,武汉 430081

摘 要:针对蒙特卡洛树搜索算法(Monte Carlo tree search,MCTS)收敛速度过慢,且在博弈过程中关键节点会出现信息丢失等问题,以中国象棋为载体,构建适用于中国象棋博弈系统的策略价值网络,提出了一种基于统计数据的并行蒙特卡洛树搜索算法(parallel Monte Carlo tree search based on statistics,SPMCTS)。将并行化的重点设置在MCTS四个步骤中最耗时的扩展和模拟步骤,有效避免了算法执行过程中的等待时差。并且引入一组新统计数据,这些数据用于在MCTS的选择步骤中修改节点的选择策略,保证在进行节点选择时获取和利用更多的可用信息,缓解信息丢失对精度造成的影响。实验结果表明,与现有并行蒙特卡洛树算法相比,SPMCTS在搜索速度上加快了约34%,且在对弈实验中,博弈胜率也能保持在80%左右。验证了SPMCTS的有效性。

关键词:蒙特卡洛树搜索;中国象棋;博弈系统;策略价值网络;并行化;统计数据
文献标志码:A **中图分类号:**TP391 doi:10.3778/j.issn.1002-8331.2401-0241

Chinese Chess Game Using Statistical Data Parallel Monte Carlo Tree Search Algorithm

ZHU Zhou, MIN Huasong
School of Information Science and Engineering, Wuhan University of Science and Technology, Wuhan 430081, China

Abstract: Aiming at the problems of slow convergence of Monte Carlo tree search (MCTS) and information loss of key nodes in the game process, a strategic value network suitable for Chinese chess game system is constructed with Chinese chess as the carrier, and a parallel Monte Carlo tree search based on statistics (SPMCTS) algorithm is proposed. The focus of parallelization is set to the most time-consuming expansion and simulation steps among the four steps of MCTS, which effectively avoids the waiting time lag during the algorithm execution. A new set of statistical data is introduced, which is used to modify the node selection strategy in the selection step of MCTS, ensuring that more available information is obtained and utilized during node selection, and alleviating the impact of information loss on accuracy. The experimental results show that compared with the existing parallel Monte Carlo tree algorithm, SPMCTS can accelerate the search speed by about 34%, and the game win rate can be maintained at about 80% in the chess experiment, which indicates the effectiveness of SPMCTS.

Key words: Monte Carlo tree search; Chinese chess; game system; strategy value network; parallelization; statistics

计算机博弈是目前人工智能的一个重要研究领域,早期人工智能的发展就是从机器下棋研究开始[1]。从1997年的“深蓝”战胜了当时国际象棋世界冠军,再到2016年Google Deepmind团队研发的AlphaGo程序战胜了韩国围棋世界冠军李世石,计算机博弈取得了举世瞩目的成就[2],极大激发了人们对于计算机博弈的关注度以及创新热情。

中国象棋作为我国传统的棋类游戏。相比于国际象棋,中国象棋的棋盘更大,且中国象棋的每个棋子都各有特殊的行棋规则,这会让中国象棋的棋局变化更复杂,由此可以看出中国象棋博弈的难度不亚于国际象棋,而且更加充满挑战[3]。此外,中国象棋也是一种完全知识博弈,即行棋双方在任意状态下都完全清楚每个棋子的现状。而且中国象棋对弈规则简单明了,胜、负、和的评定标准单一,属于二人零和博弈(即博弈双方的获利和损失相加和总是为零),不会有任何偶然性的结局[4]。

目前,中国象棋计算机博弈的研究主要是通过提高博弈程序的搜索效率以及优化局面评估函数,从而提高计算机的象棋博弈水平。Tao等人[5]通过引入深度迭代的方法,丢弃无价值的节点,有效减少了搜索的节点

基金项目:国家重点研发计划(2022YFB4700400);国家自然科学基金(62073249);湖北省重点研发计划(2023BBB011)。
作者简介:朱舟(1996—),男,硕士,研究方向为计算机博弈、深度强化学习;闵华松(1969—),通信作者,男,教授,研究方向为协作型工业机器人、嵌入式系统,E-mail:mhuasong@wust.edu.cn。
收稿日期:2024-01-17 **修回日期:**2024-03-25 **文章编号:**1002-8331(2024)23-0340-09

超星·期刊

数。Chen等人[6]通过自动调整特征权重,改进了局面评估函数,使博弈胜率得到有效的提升。He等人[7]通过引入Q学习方法和评价函数,有效地提高了中国象棋的博弈能力。

近年来,深度学习算法受到了众多国内外学者的关注,计算机博弈方面也取得了飞速的发展。与传统博弈不同,应用了深度学习的计算机博弈主要使用蒙特卡洛树搜索算法(MCTS)和深度卷积神经网络相结合,并且通过神经网络来学习现有经验知识,以此为基础改进其中的参数,输出胜率最大的走法[8]。

现流行应用深度神经网络的三种并行MCTS算法分别为:叶并行化(leaf parallelization)[9]、根并行化(root parallelization)[10]、树并行化(tree parallelization)[11]。三种并行方法各有其优缺点,叶并行化最容易实现但缺乏对节点的探索;根并行化减少了线程通信但同时也减少了每个独立树上的博弈次数;树并行化能够最小化通信开销但与之对应的是搜索开销比较大[12]。Milewicz等人[13]表明,现阶段并行MCTS最大问题是每次进行搜索迭代时都需要来自先前搜索树中的信息,但是在这个过程中总会出现信息的丢失,这些信息的丢失可能会使关键节点并不能访问足够多的次数来给出合理的估计,导致性能下降。

为有效解决上述问题,本文基于深度强化学习的思想,构建了一种适用于中国象棋的策略价值网络模型,定义了网络的输入、输出特征表示、神经网络的结构以及模型的损失函数。主要贡献如下:

(1)针对蒙特卡洛树收敛速度过慢问题,将MCTS进行有效的并行化。在并行过程中实现了将并行方法主要用于蒙特卡洛树搜索算法中耗时较多的扩展和模拟步骤,有效地缩短了搜索时间。

(2)针对并行化存在的信息丢失问题,提出了一种基于统计数据的并行蒙特卡洛树搜索算法,称为SPMCTS。通过引入一组特殊数据,该数据用来记录正在进行模拟还没有完成的线程数量。本文将这组统计数据应用在选择步骤中修改选择策略值,保证在节点选择中获取和利用更多的可用信息,确保节点选择的精确度。

1　相关理论

1.1　蒙特卡洛树搜索

蒙特卡洛树搜索(MCTS)是一种构建搜索树的方法,通过依次向树中添加单个节点,每个节点表示某种状态,然后从新添加的节点开始进行随机选择,直到到达终端状态,之后对各个节点进行评分,最后根据节点的优劣情况再进行选择步骤[14]。

蒙特卡洛树搜索算法主要是通过模拟双方下棋来预测棋局走向,并反馈更新信息,经过上千次搜索后,得到胜率最大的走子策略[15]。在对局期间,MCTS算法使用现有的神经网络进行评估,根据当前所有节点的评估值获得当前盘面中较优的行棋方法,并不断循环使用该方法进行自我对弈直至产生胜负后或达到规定的搜索上限[16]。对弈结束后每步棋的局面状态、动作概率向量和胜率被保存下来,并作为棋谱数据来训练神经网络。

MCTS算法主要包括选择、扩展、模拟、反向传播四个步骤,算法执行过程如图1所示。

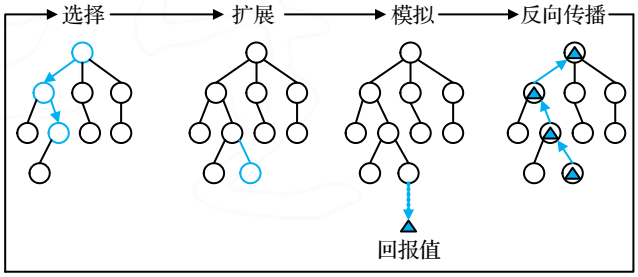


图1　MCTS算法执行过程

Fig.1　MCTS algorithm execution process

步骤1　选择。从根节点开始,选择 $Q(s,a)+U(s,a)$ 最大的节点,并将该子节点作为下一次选择的起点,直到到达叶子节点[17]。其中 $Q(s,a)$ 和 $U(s,a)$ 分别为节点的平均行动价值和UCB值,$Q(s,a)$ 和 $U(s,a)$ 的计算公式如下:

$$U(s,a)=CP(s,a)\frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)} \tag{1}$$

$$Q(s_t,a_t)=W(s_t,a_t)/N(s_t,a_t) \tag{2}$$

其中,C 是一个决定探索水平的常数;s 表示当前的棋局状态;s_t 表示搜索树中当次被遍历路径上节点对应的棋局状态;a 为子节点;a_t 表示搜索树中当次被遍历路径上节点对应的棋局状态下选择的子节点;b 代表父节点;$P(s,a)$ 为当前状态下节点 a 的先验概率;$N(s,a)$ 为当前状态下节点 a 被选中的次数;$W(s,a)$ 为当前状态下节点 a 的总收益。

步骤2　扩展。对步骤1中选择的叶子节点进行扩展,根据当前叶子节点的棋局状态,生成将选中的叶子节点所允许的所有子节点作为新的叶子节点,然后调用神经网络评估当前局面,输出各个子节点的合法位置 s_L 和落子概率 p_a。将扩展得到的新子节点的初始信息设置如式(3)所示:

$$\begin{cases} N(s_L,a)=0, W(s_L,a)=0 \\ Q(s_L,a)=0, P(s_L,a)=p_a \end{cases} \tag{3}$$

步骤3　模拟。扩展完成后,对该节点进行模拟,即重复步骤1、2,选择 $Q(s,a)+U(s,a)$ 得分最高的节点作为模拟下一步的落子,如果该节点不是棋局结束状态,则继续对其进行扩展,直到棋局状态已经分出胜负或达到本文设定好的搜索次数,此时博弈结束并将胜负信息

超星·期刊

作为节点的价值 v。

步骤4 反向传播。模拟完成后,将节点的价值反向传给父节点,并一直向上更新祖先节点的信息,直至根节点。被选中的节点总的行动价值加 v,访问次数加1,并重新计算平均行动价值。更新的公式如式(4)~式(6)所示:

$$W(s_t,a_t)=W(s_t,a_t)+v \tag{4}$$

$$N(s_t,a_t)=N(s_t,a_t)+1 \tag{5}$$

$$Q(s_t,a_t)=W(s_t,a_t)/N(s_t,a_t) \tag{6}$$

需要注意的是由于搜索树中相邻两层的落子双方是进行对抗的,一方胜利就意味着另一方的失败。所以相邻两层的节点价值互为相反数。

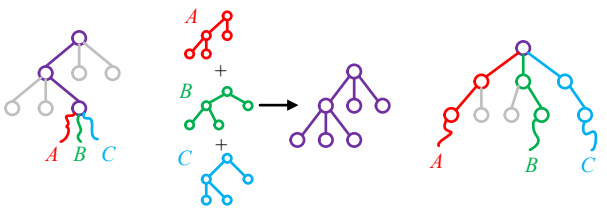
在行一步棋之前,神经网络会进行上千次的搜索,每次搜索都会进行上述四个步骤。达到设定的搜索次数后,会根据式(7)进行落子:

$$\pi(a|s)=N(s,a)^{1/\tau}/\sum_b N(s,b)^{1/\tau} \tag{7}$$

其中,$\pi(a|s)$ 表示当前节点下,通过蒙特卡洛树搜索得到的每种策略被选择的实际概率;τ 为温度参数,用来控制行棋过程的探索程度。

1.2 相关并行蒙特卡洛树搜索

由于蒙特卡洛树算法庞大的搜索空间使得算法在进行决策时耗费了相当长的计算时间。因此,神经网络中使用的都是多线程的并行蒙特卡洛树搜索算法[18]。现阶段并行蒙特卡洛树搜索主要分为三种:叶并行化、根并行化和树并行化,如图2所示。



(a)叶并行化　(b)根并行化　(c)树并行化

图2　并行蒙特卡洛树搜索算法

Fig.2　Parallel Monte Carlo tree search algorithm

(1)叶并行化:如图2(a)所示,只有一个线程遍历搜索树,并在到达叶节点时将一个或多个节点添加到搜索树中。从叶子节点开始,由多个线程进行独立的模拟,直到达到终局状态[19]。之后,将所有模拟得到的结果结合起来,沿着遍历的分支进行反向传播。Mirsoleimani等人[20]指出叶并行化的缺点是模拟时间不可预测,因为在进行反向传播之前必须要同步线程,即所有进行模拟的线程必须等待速度最慢的线程完成后才能进行反向传播。

(2)根并行化:如图2(b)所示,由并行构建的多个MCTS组成,每个MCTS都有一个线程。线程之间不共享信息,当模拟次数达到本文设置的次数后,每个独立树的所有根、子节点都将与其对应的其他树的节点进行合并。Victor等人[21]认为虽然这些树是从同一个节点开始创建的,但MCTS的随机选择意味着每棵树的形成都不同于其他树,且进行树的合并时,通常是将所有树的每个可能的节点的所有回报值相加。这种情况下,一个节点的综合回报是每个独立的树相加的总和。

(3)树并行化:如图2(c)所示,使用多个线程在同一个搜索树中进行搜索。每个线程同时执行MCTS的四个阶段。为了防止多个线程同时访问一个节点导致数据损坏,会将互斥锁放置在线程当前正在使用的节点上。但是添加锁后就意味着其他线程必须等到锁被释放才能继续访问相关节点,出现所谓的“等待时差”,在这期间会使线程处于空闲状态,对并行性能造成影响。这个问题的一个解决方案是引入“虚拟损失”。当线程选中一个节点时,会将该节点的回报值减去虚拟损失,当该节点反向传播结束后,再将虚拟损失删除,还原该节点的真实回报[22]。但Mirsoleimani等人[23]经过实验证明虚拟损失的方法搜索开销比较大,实际上对并行化的性能和效率并没有得到很好的改善。

2 方法

2.1 神经网络结构设计

策略价值网络结构如图3所示,该网络是将原先独立的策略网络和价值网络合并成了一个深度神经网络[24]。在该神经网络中,从输入层到中间层的权重是完全共享的,最后的输出阶段分成了策略网络输出和价值网络输出。该网络的输入特征为9个10×9的二值特征平面来描述当前棋局状态,10×9代表象棋的棋盘盘面,由于象棋有7种不同的棋子,因此本文用前7个平面表示对弈双方的棋子位置,第8个平面表示对方玩家最近一步的落子位置,而第9个平面用来表示当前玩家是否为先手玩家。

由于象棋棋盘的特征也非常多,所以输入特征通过卷积块后先将通道数扩充到256,再利用7个残差块对输入特征进行更深层次的提取,每个残差块都连续进行了两次卷积,之后使用ReLu激活函数分成策略端和价值端两个输出。在策略端方面,先使用16个1×1的filter进行降维,再接一个全连接层、内有2 086个神经元,最后使用softmax函数直接输出棋盘上所有可能的走子概率。在价值端方面,结构同样类似,最后使用tanh函数输出[−1,1]之间的值来表示胜率预测值。

策略网络的输入是当前棋盘局面的状态信息 s,输出则是当前局面进行可行性落子的概率分布 p。之后进行自对弈收集所需要的训练集,保存每个棋盘局面的状态信息 s,通过MCTS迭代完成后得到相对应每个局面下的动作选择概率分布 p。最后通过神经网络的训练,策略网络能够根据当前棋盘局面预测出更为准确的

超星·期刊

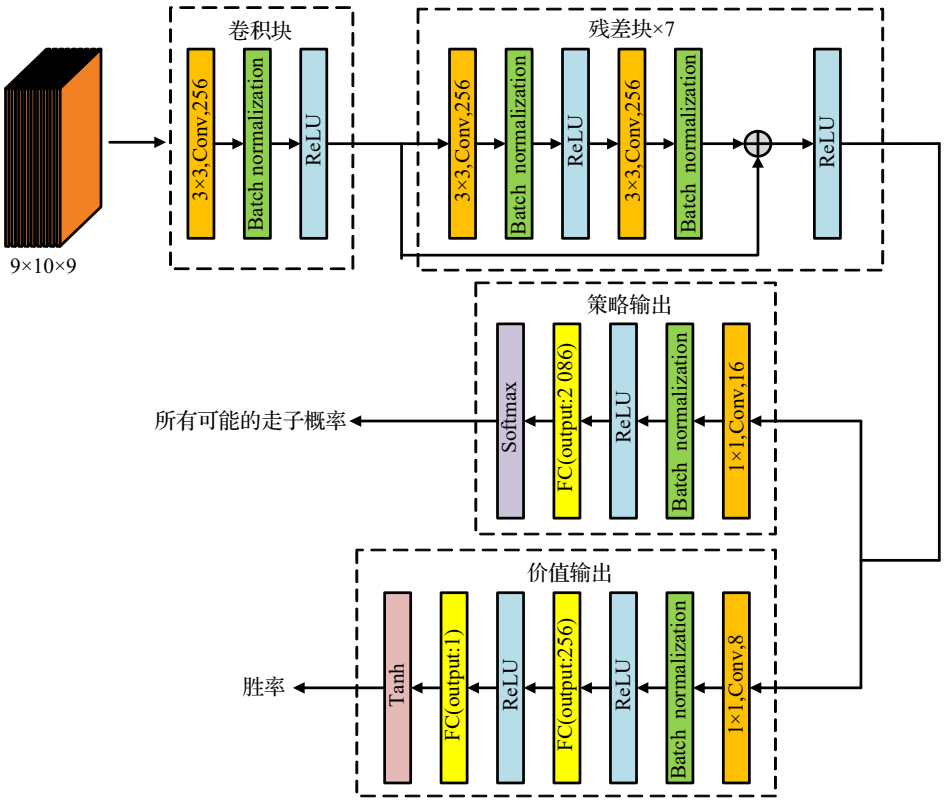


图3　策略价值网络结构图

Fig.3　Strategy value network structure chart

落子概率分布。

价值网络的输入同样是当前棋盘局面的状态信息 s ,输出则是对当前局面的评估分数 v 。通过进行自对弈,可知每一局对弈结束后的胜负信息,并用 z 对真实的对局结果进行标记,以此作为训练集进行网络参数的训练,目的是使价值网络能够根据当前棋盘局面预测出更接近于真实结果的评估分数。

根据以上分析,本文模型的策略网络主要反映的是神经网络预测的走子概率 p 与通过MCTS采样得出的走子概率 p 的相似度,而交叉熵能够衡量两个分布的异同程度,所以选择交叉熵作为损失函数;而价值网络是通过减少真实的对弈结果 z 与神经网络估计的胜率值 v 的胜率预测误差,是一个回归问题,因此选用均方误差作为损失函数。所以本文模型的策略价值网络的损失函数为:

$$L=(z-v)^2-\pi^{\mathrm{T}}\ln p+c\left\|\theta^2\right\| \tag{8}$$

损失函数主要由三部分组成:第一部分是用来评估神经网络输出的胜负结果与真实对局结果之间的差异;第二部分是交叉熵损失函数,用于评估神经网络输出的落子概率和MCTS输出的概率的差异;第三部分是正则化项, c 为控制L2正则化项权重调整的参数,用来防止过拟合现象, θ 是神经网络的参数[25]。

2.2　基于统计数据的并行蒙特卡洛树搜索

在并行MCTS中,除了想要将MCTS进行有效的并行化之外,还需要减少信息损失带来的问题。本文提出的SPMCTS主要从两方面进行改进,第一部分是对并行的方法进行改进,加快MCTS的搜索速度;第二部分则是对选择步骤中的策略问题进行改进,避免不同的进程重复选择同一个节点,增加探索多样性的同时保证结果的准确性。

MCTS总是顺序地执行选择、扩展、模拟、反向传播四个基本步骤,而并行蒙特卡洛树搜索的主要思想是利用线程的可并发性,每个线程都按顺序执行MCTS的四个步骤,执行完成后将结果反馈给主进程,由主进程进行节点信息的更新。这样虽然能加快MCTS的搜索速度,但实际上还是存在一些问题。因为这四个步骤所消耗的时间是不相同的,选择步骤只需要选中价值最高的节点即可,反向传播步骤是将子节点的信息反馈给根节点,而扩展和模拟步骤则是需要将棋局状态进行展开,一直到分出胜负为止。所以,与选择和反向传播步骤相比,扩展和模拟步骤要耗时得多。因此本文将并行化重点放在扩展和模拟这两个步骤中。具体来说,主程序重复执行选择、扩展、模拟、反向传播步骤,直到达到预定义的模拟次数为终止。在每次执行期间,它会根据选择步骤中的策略进行节点的选择,根据节点的状态将它们分别分配到扩展和模拟线程中,若扩展线程全被占用,将完成扩展步骤的节点分配给模拟线程,执行模拟步骤,模拟线程全部被占用后,等待模拟步骤完成,最后进

超星·期刊

行反向传播,更新各个节点的信息,进行新一轮的蒙特卡洛树搜索。

由于线程进行选择节点时需要访问最新的统计数据,在并行蒙特卡洛树搜索过程中,当多个线程同时执行MCTS的四个步骤,某一线程在进行选择步骤时,其他线程可能正在进行扩展和模拟步骤,并没有返回结果,导致主进程无法对节点的信息进行更新,这样会使新启动的线程在选择阶段只能看到过时且类似的信息,出现重复选择同一节点的情况,使本应该被选择的节点出现信息丢失问题。针对这个问题,本文发现如果有节点被选中,那么在不久的将来一定会得到它的模拟返回值,因此可以看作其节点已经被选择过了。所以只要线程启动新的扩展,就可以使用最新的数据信息。基于这个思路,本文引入了一组新的统计数据 T_s 加入到选择策略中,T_s 表示已经进行了扩展操作,但还未完成模拟步骤的线程数量。修改后的选择策略如下式所示:

$$U(s,a)=CP(s,a)\frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)+T_s} \tag{9}$$

$$Q(s_t,a_t)=W(s_t,a_t)/N(s_t,a_t)+T_s \tag{10}$$

上述修改后的选择节点策略是,当有 T_s 个线程正在进行模拟步骤时,节点 a 的策略值 $Q(s,a)+U(s,a)$ 会在模拟完成前缩小,可以减缓该节点被选中后到完成模拟之前统计数据不变的问题。相当于对被选中节点的策略值做了一个提前的衰减,因此可以保证最新启动的线程获得更新后的策略值,由于策略值发生改变,新的线程在选择阶段会较大概率地选择其他没被探索过的节点,增加了并行化探索的多样性。SPMCTS的整体架构如图4所示。

这样修改后的好处是:相比于原策略,多增加了一次更新节点策略值的过程。原方法是要按照选择、扩展、模拟、反向传播四个步骤完成后,再对选中的节点进行统计数据的更新;而SPMCTS是在通过选择策略选中节点后,对该节点进行扩展,再将扩展后的节点分配给模拟线程,统计此时正在进行模拟但还没完成的线程数量,将该数据传回到选择策略中,更新一次节点的策略值,这样新启动的线程就会根据最新的节点值进行选择,等模拟完成后,进行反向传播,再更新一次节点的值,在一次过程中完成了两次策略值的更新,有效减少了在线程并行化过程中信息丢失的问题。

相较于1.2节提到了三种流行的并行算法,本文通过引入额外的统计数据 T_s 的方法能实现更好的探索和利用之间的平衡。首先,LeafP因将所有的线程都用到了同一个节点下进行模拟,在探索方面缺乏多样性,而SPMCTS是通过对选择阶段策略进行修改,增加了探索更多节点的可能性;其次,RootP是通过让线程独立地执行多个树搜索,但这不可避免地降低了每个树的博弈次数,从而降低UCT策略的准确性,而SPMCTS是在同一颗搜索树上进行博弈,避免了这个问题;最后,TreeP虽然使用了“虚拟损失”来增加探索多样性,但这种在选择策略中硬加惩罚项可能会导致利用的失败[26],如即使知道这个被选中的节点是最佳的,但是其他的线程会因为虚拟损失的影响而不再访问该节点,相比之下SPMCTS同样使用 T_s 来“惩罚”那些正在进行模拟的节点来鼓励探索,但它会允许多个线程同时访问回报值最大的节点,因为当某个节点的访问次数变大时,本文所添加的 T_s 就会变得忽略不计,即这种“惩罚”就会消失。

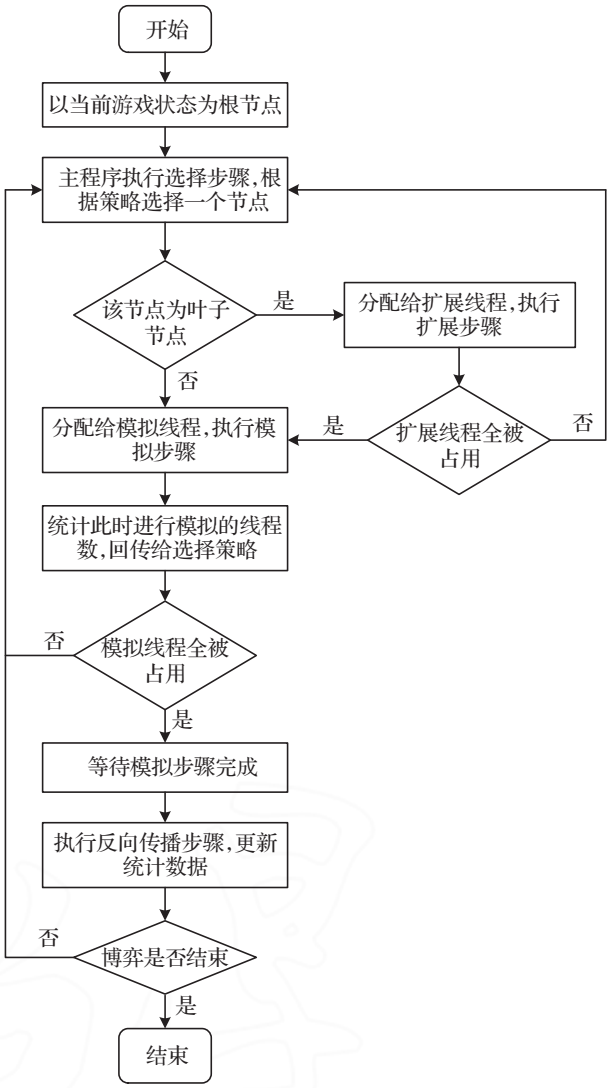


图4　SPMCTS流程图

Fig.4　Flow chart of SPMCTS

3　实验结果与分析

本章主要是将本文提出的算法对比现流行的三种并行MCTS算法(叶并行化LeafP、根并行化RootP、树并行化TreeP)进行多组实验,来验证所提出的SPMCTS算法的性能。

超星·期刊

3.1　实验准备

实验参考AlphaZero的思想,该方法在不需要数据集的情况下通过模型进行自对弈的方式产生数据进行训练。将设计的应用于中国象棋的神经网络分别与不同的并行MCTS算法相结合,进行网络训练,得到对应的神经网络模型,收集在训练过程中产生的相关数据,并利用收集到的数据和模型来进行对比实验。收集的数据包括:模型训练时产生的损失值、信息熵,模型进行一次搜索的时间。收集到的模型用于对弈实验,记录不同模型对弈的胜率。

本文实验的环境配置如下:开发工具为Pycharm,软件环境为Pytorch1.10.2。由于模型训练过程中需要大量的计算资源,本文采用两台计算机分别运行训练程序和博弈程序:一台Linux内核的服务器,Intel Xeon CPU E5-2678,运行频率为2.50 GHz,128 GB内存,两块NVIDIA GeForce RTX 2080 Ti的显卡,用于进行神经网络模型的训练,收集训练过程中产生的数据和模型;一台Window 10操作系统的计算机,CPU为Intel® Core™ i5-10200H,主频2.40 GHz,16 GB内存,显卡为NVIDIA GeForce GTX 1650,运行博弈程序,用于进行对弈实验。

3.2　实验结果

为了更有效对比分析不同版本的并行蒙特卡洛树搜索算法的性能,实验在后续的模型训练中将每个版本的并行MCTS的模拟次数统一设置为600。这样一是可以更快地完成搜索,二是MCTS经过600次模拟后也具备了一定的博弈能力。能更好利用有限计算机资源,利于实验的进行。

3.2.1　搜索速度

在相同的实验环境下,通过设定不同的模拟次数,记录下SPMCTS与其他另外三种并行蒙特卡洛树搜索算法进行一次搜索后所用的平均时间,从时间的大小可以很容易看清不同并行算法的搜索速度。本次实验设定的模拟次数分别为100、200、300、500、800、1 000、1 200。四种算法进行一次搜索耗时如图5所示。

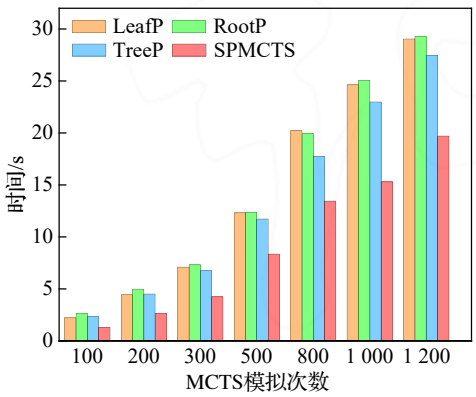


图5　不同并行MCTS的搜索速度

Fig.5　Search speed of different parallel MCTS

从图中可以明显地看出,在相同的模拟次数下,SPMCTS的搜索速度对比其他三种算法有很明显的提升,在1 200次的模拟次数下,LeafP搜索一次时间约为29 s,RootP搜索一次时间约为29.3 s,TreeP搜索一次时间约为27.4 s,而SPMCTS搜索一次时间约为19.7 s,相比另外三种算法搜索一次所用时间加快了约34%。这说明本文在搜索过程中将耗时最多的扩展和模拟的步骤集中并行的方法是有效的,能够缩短博弈树的搜索时间。

3.2.2　损失值

损失函数是让策略价值网络输出的当前局面下每个可行动作的概率更加接近蒙特卡洛树搜索输出的概率,让策略价值网络输出的局面评分能更准确地预测真实结果[27]。损失值越小,神经网络的预测结果与蒙特卡洛树搜索的输出结果越接近,模型预测值与真实值差别越小,模型的质量越高。

本文选取不同并行算法的前2 000次训练过程中产生的损失值进行对比实验。4种并行算法模型的损失值变化情况如图6所示。

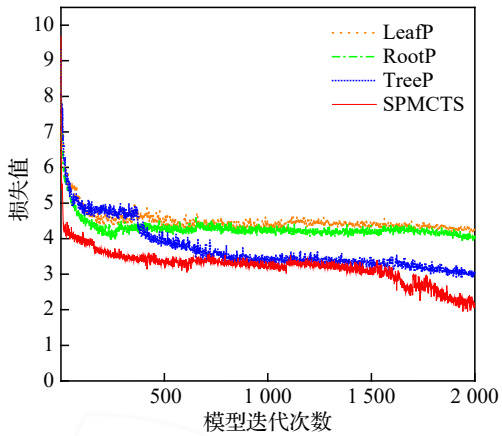


图6　不同并行MCTS的损失值

Fig.6　Loss of different parallel MCTS

从图中可以看出,在不同并行算法训练的模型损失值对比曲线图中,SPMCTS训练出的模型损失值相比于其他3种算法模型更低。不同算法模型训练到2 000次后,LeafP的损失值约为4.2,RootP的损失值约为4.0,TreeP的损失值约为2.9,而SPMCTS的损失值约为2.1,明显低于其他3种并行算法,说明本文的算法训练出来的模型对棋局状态的判断更加准确,模型的质量更高,符合预期。

3.2.3　信息熵

信息熵描述的是策略价值网络输出的落子概率分布[28]。刚开始进行训练时,策略价值网络基本上是随机落子,所以相对来说信息熵会很大,随着训练的进行,神经网络会慢慢学会在不同局面下寻找落子概率更大的位置,即落子概率不再均匀,会有很强的偏向性,信息熵就会逐渐变小。所以,信息熵越小,越能帮助蒙特卡洛

超星·期刊

树搜索在进行搜索过程中在更有潜力的节点进行更多的模拟,这样训练出来的模型效果也越好。

实验同样选取了SPMCTS与LeafP、RootP、TreeP 3种并行算法的前2 000次训练过程中产生的信息熵进行对比。4种并行算法模型的信息熵变化情况如图7所示。

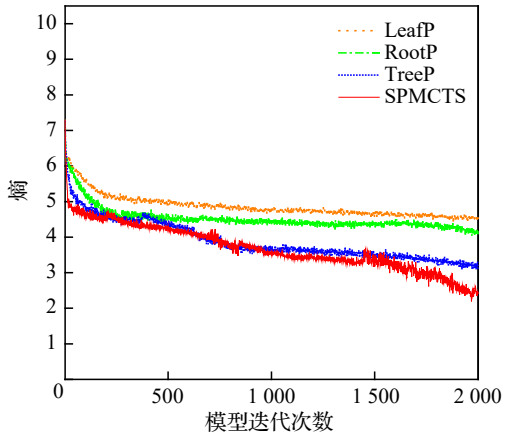


图7　不同并行MCTS的信息熵

Fig.7　Entropy of different parallel MCTS

从图7可以看出,SPMCTS算法在训练过程中模型的信息熵明显低于其余3种并行算法。训练次数到达2 000次时,LeafP的信息熵约为4.5,RootP的信息熵约为4.1,TreeP的信息熵约为3.2,SPMCTS的信息熵约为2.4。这说明本文的算法相较于另外3种并行算法,对那些潜力高的节点有更强的偏向性,使本文能够选中胜率最大的节点,从侧面反映出本文训练出来的模型质量更高。

3.2.4　对弈胜率

实验主要通过SPMCTS与另外3种并行算法分别进行对弈,统计胜率,验证算法的博弈能力。由于计算机资源的有限,本文选取4种算法前2 000次训练得到的模型进行实验。另外为了方便进行对比分析,每训练400次选择一次模型,因为模型经过400次训练后相比前一代模型博弈能力会得到提升。

实验选取模拟次数为600,训练次数分别为400、800、1 200、1 600、2 000的模型,局数设定为60,对比的两种算法分别进行30轮的先手和后手,本文设定了先手出现平局判负,这样结果就只会有胜、负两种情况,方便本文进行胜率的统计。实验统计的结果分别如表1、表2、表3所示。

表1　SPMCTS与LeafP对弈结果

Table 1　Results of SPMCTS-vs-LeafP

实验组	训练次数	胜:负	胜率/%
1	400	36:24	60.0
2	800	41:19	68.3
3	1 200	43:17	71.7
4	1 600	47:13	78.3
5	2 000	52:8	86.7

表2　SPMCTS与RootP对弈结果

Table 2　Results of SPMCTS-vs-RootP

实验组	训练次数	胜:负	胜率/%
1	400	35:25	58.3
2	800	38:22	63.3
3	1 200	43:17	71.7
4	1 600	46:14	76.7
5	2 000	50:10	83.3

表3　SPMCTS与TreeP对弈结果

Table 3　Results of SPMCTS-vs-TreeP

实验组	训练次数	胜:负	胜率/%
1	400	32:28	53.3
2	800	37:23	61.7
3	1 200	40:20	66.7
4	1 600	44:17	73.3
5	2 000	46:14	76.7

另外,为了更好看清模型对弈胜率的变化情况,本文将3组实验结果绘制在同一张图里,如图8所示。从图表可以看出,SPMCTS与另外3种并行算法进行对弈时,胜率有明显的优势。当模型训练次数到达2 000时,SPMCTS与LeafP与RootP对弈胜率分别为86.7%和83.3%,与TreeP对弈的胜率也达到了76.7%。并且从图8中也可以看出,随着训练次数的增加,SPMCTS对抗其他并行算法的胜率也在逐步提升。这一方面说明本文对蒙特卡洛树搜索算法在选择阶段的策略进行的改进,确实有助于算法博弈能力的提升;另一方面也可以从侧面反映出SPMCTS较于另外3种并行算法在搜索多样性上更具有优势,因为SPMCTS的对弈胜率更高就意味着SPMCTS在搜索过程中能够对博弈树中更多的节点进行搜索,从而使每个节点的实际价值更加准确,并从中选择出价值最大的节点。

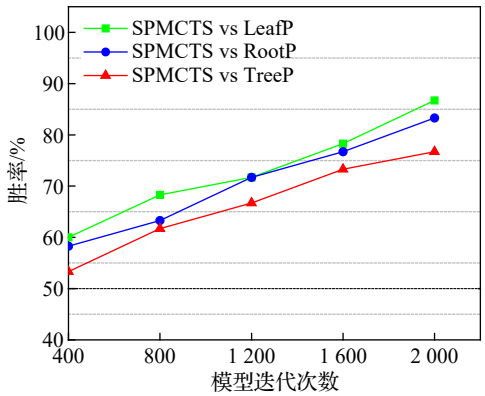


图8　SPMCTS胜率

Fig.8　Winning rate of SPMCTS

3.2.5　棋力测评

由于本文的模型是经过神经网络训练得到的,所以在训练过程中的初期、中期以及后期的明显差异使模型同样表现出了不同的行棋风格。因此,本文使用新旧模型进行对弈,验证模型的棋力是否随训练次数的增加而

超星·期刊

有明显提升。

实验中选取训练次数为8 000的模型(最新训练模型)分别与训练次数为2 000、4 000、6 000的模型进行对弈,局数设定为30,平局由双方超过30步未吃子产生。

对弈情况如表4所示,当8 000代模型与2 000代及4 000代模型对弈时,虽然出现了较大概率的平局,但是没有败绩。当8 000代模型与6 000代模型对弈时,虽有一定的胜率但产生了败绩,同样平局比例占绝大多数,出现这种情况的主要原因是进行对弈的模型训练间隔较小,棋力差距并不明显。

表4　不同模型对弈结果

Table 4　Results of different training models

实验组	对弈模型	胜:和:负
1	8 000 VS 2 000	13:17:0
2	8 000 VS 4 000	9:21:0
3	8 000 VS 6 000	10:18:2

本文还与现阶段比较流行的象棋AI算法GGZERO进行了比较,GGZERO是一个关于中国象棋引擎的公益项目,它是由国际象棋Leela Zero项目改编而来,同样应用了深度强化学习的方法来提升棋力。本文将GGZERO的模型同样训练到8 000次,与本文算法最新训练次数相同。实验过程中选取了两种算法分别训练了2 000次和8 000次的模型进行对弈,选择训练2 000次和8 000次的模型是因为经过2 000次训练后模型已经具备了初步的博弈能力,而8 000次的训练模型是本文能在有限的计算机资源情况下得到的最好的模型,并且模型训练间隔越大,模型博弈能力的提升也就越明显。局数设定为80局,双方各40局先手和后手,对弈结果如表5所示。

表5　SPMCTS与GGZERO对弈结果

Table 5　Results of SPMCTS-vs-GGZERO

实验组	训练次数	胜:和:负
1	2 000	8:67:5
2	8 000	26:43:11

从表中可以看出,SPMCTS和GGZERO在训练到2 000次时,双方各有胜负,且平局占绝大多数,这是因为在训练初期,通过自对弈产生的数据量是非常有限的,模型都是随机下棋,没有策略偏向性,棋力差距并不明显。而当训练次数到达8 000次后,模型的博弈能力得到了提升,SPMCTS的胜率明显高于GGZERO。证明了本文算法的可行性。

由于计算机硬件设备的原因,本文算法产生的模型最终棋力也很难达到专家水平,并且在现有的象棋软件(天天象棋)上进行了评级测试,也只达到了入门级别。但从表5可以看出,随着训练次数不断增加,最新模型的棋力还是有了明显的提升。

4　结语

本文在深度强化学习的理论基础上,设计了适用于中国象棋博弈系统的深度神经网络结构,并且考虑到蒙特卡洛树搜索收敛过慢以及迭代过程中信息丢失等问题,提出了基于统计数据的并行蒙特卡洛树搜索算法,通过有效的并行化和修改选择策略,在加快搜索效率的同时保证博弈胜率。最后,通过实验验证了相比于其他算法,SPMCTS在搜索速度和博弈胜率上均有明显的优势。本文提出的SPMCTS虽然只在中国象棋上进行了测试,但本质上是对蒙特卡洛树搜索算法进行的改进,对于五子棋、围棋等其他棋类游戏同样具有借鉴价值。未来的研究重点是优化神经网络结构,探究不同的网络参数对模型棋力的影响,进一步提升模型的博弈水平。

参考文献:

[1] GRANTER S R, BECK A H, PAPKE D J. AlphaGo, deep learning, and the future of the human microscopist[J]. Archives of Pathology & Laboratory Medicine, 2017, 141(5): 619-621.

[2] SILVER D, HUANG A, MADDISON C J, et al. Mastering the game of Go with deep neural networks and tree search [J]. Nature, 2016, 529(7587): 484-489.

[3] HUANG C M, GUO J H, SU K L. Based on short motion paths and artificial intelligence method for Chinese chess game[J]. Journal of Robotics Networking and Artificial Life, 2017, 4(2): 154-157.

[4] WANG F, HOU X, DUAN Z, et al. The perceptual differences between experienced Chinese chess players and novices: evidence from eye movement[J]. Acta Psychologica Sinica, 2016, 48(5): 457.

[5] TAO J, WU G, PAN X. Design and improvement of the pruning algorithm of the Chinese chess in the computer games[J]. The Journal of Engineering, 2020(13): 426-428.

[6] CHEN J C, TSENG W J, WU I C, et al. Comparison training for computer Chinese chess[J]. IEEE Transactions on Games, 2020, 12(2): 169-176.

[7] HE W, ZHAO W, JIANG Y. Application of Q-learningand RBF network in Chinese chess game system[J]. IOP Conference Series: Materials Science and Engineering, 2019, 677(2): 022101.

[8] SILVER D, SCHRITTWIESER J, SIMONYANK, et al. Mastering the game of Go without human knowledge[J]. Nature, 2017, 550(7676): 354-359.

[9] GOLDWASER A, THIELSCHER M. Deep reinforcement learning for general game playing[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2020: 1701-1708.

[10] SOEJIMA Y, KISHIMOTO A, WATANABEO. Evaluating root parallelization in Go[J]. IEEE Transactions on Computational Intelligence & AI in Games, 2011, 2(4): 278-287.

超星·期刊

[11] LIU S, CAO J, WANG Y, et al. Self-play reinforcement learning with comprehensive critic in computer games[J]. Neurocomputing, 2021, 449(18): 207-213.
[12] FAN S, ZHANG S, LIU J, et al. Power converter circuit design automation using parallel Monte Carlo tree search [J]. ACM Transactions on Design Automation of Electronic Systems, 2023, 28(2): 17-33.
[13] MILEWICZ R M, POULDING S. Scalable parallel model checking via Monte-Carlo tree search[J]. ACM SIGSOFT Software Engineering Notes, 2018, 42(4): 1-5.
[14] BROWNE C B, POWLEY E, WHITEHOUSE D, et al. A survey of Monte Carlo tree search methods[J]. IEEE Transactions on Computational Intelligence & AI in Games, 2012, 4(1): 1-43.
[15] ZHANG J, SUN X, ZHANG D, et al. Fittest survival: an enhancement mechanism for Monte Carlo tree search[J]. International Journal of Bio-Inspired Computation, 2021, 18(2): 122-130.
[16] BORY P. Deep new: the shifting narratives of artificial intelligence from deep blue to AlphaGo[J]. Convergence: The International Journal of Research into New Media Technologies, 2019, 25(4): 627-642.
[17] ALBA C, VICENTE G, EDWARD R N, et al. Improving Monte Carlo tree search with artificial neural networks without heuristics[J]. Applied Sciences, 2021, 11(5): 2056.
[18] MIRSOLEIMANI S A, HERIK J V D, PLAAT A, et al. Pipeline pattern for parallel MCTS[C]//Proceedings of the 10th International Conference on Agents and Artificial Intelligence, 2018: 614-621.
[19] STEINMETZ E S, GINI M. More trees or larger trees: parallelizing Monte Carlo tree search[J]. IEEE Transactions on Games, 2021, 13(3): 315-320.
[20] MIRSOLEIMANI S A, HERIK J V D, PLAAT A, et al. A lock-free algorithm for parallel MCTS[C]//Proceedings of the 10th International Conference on Agents and Artificial Intelligence, 2018: 589-598.
[21] VICTOR G, ROLANDO E N, VICENTE G, et al. Monte Carlo tree search as a tool for self-learning and teaching people to play complete information boardgames[J]. Electronics, 2021, 10(21): 2609.
[22] SILVER D, HUBERT T, SCHRITTWIESER J, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play[J]. Science, 2018, 362(6419): 1140-1144.
[23] MIRSOLEIMANI S A, HERIK J V D, PLAAT A, et al. An analysis of virtual loss in parallel MCTS[C]//Proceedings of the 9th International Conference on Agents and Artificial Intelligence, 2017: 648-652.
[24] ZHU X, LUO Y, LIU A, et al. A deep reinforcement learning-based resource management game in vehicular edge computing[J]. IEEE Transactions on Intelligent Transportation Systems, 2022, 23(3): 2422-2433.
[25] WANG L, ZHAO Y, JINNAI Y, et al. Neural architecture search using deep neural networks and Monte Carlo tree search[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2020: 9983-9991.
[26] LIU P S, ZHOU J Z, LV J C. Exploring the first-movebalance point of Go-Moku based on reinforcement learning and Monte Carlo tree search[J]. Knowledge-Based Systems, 2023, 261(15): 110207.
[27] DONG P, LIU H C, LEI X. Monte Carlo tree search based non-coplanar trajectory design for station parameter optimized radiation therapy (SPORT)[J]. Physics in Medicine and Biology, 2018, 63(13): 135014.
[28] WANG Y F, WEI Y L, HUANG X L, et al. Robot navigation with predictive capabilities using graph learning and Monte Carlo tree search[J]. Proceedings of the Institution of Mechanical Engineers, 2023, 237(5): 805-814.